ON THE PERFORAMANCE OF LINEAR SYSTEM SOLVERS IN ALTERNATING LEAST SQUARES FOR COLLABORATIVE FILTERING

Franniel Luigi Hilario Brian Gabriel Magbanua Fiona Ventura
Jermaine Pasamba Mark Oliver Medina

College of Science, Bachelor of Science in Mathematics With Specialization in Computer Science, Bulacan State University, Malolos, Bulacan

ABSTRACT

Todo

INTRODUCTION

Recommender systems have become an essential component of modern information platforms which assist users to find relevant content in environments that contain extensive item collections and users who have restricted viewing time. The system uses past user behavior data to identify user preferences which it uses to generate customized content suggestions for e-commerce platforms and streaming services and social media networks (Ricci et al., 2015). Collaborative filtering stands out among recommendation methods because it can successfully identify user preferences based on user-item interaction patterns which it derives from data analysis without needing direct item attribute knowledge (Koren et al., 2009).

The Netflix Prize competition created a key development for collaborative filtering by pushing researchers to build recommendation systems which could handle large datasets while providing precise predictions. Matrix factorization emerged as the leading framework from this era because it uses a user-item rating matrix which requires two low-rank latent factor matrices to create user preference and item characteristic representations (Koren et al., 2009; Takács et al., 2009). The system uses this formulation to reveal hidden patterns between users and items while handling computational demands of extensive datasets.

The matrix factorization computation field has made Alternating Least Squares (ALS) into a common method because of its basic operation design which delivers strong results and works well with parallel processing. The ALS method breaks down its optimization task into two parts which need to change user and item latent factor matrices. The algorithm needs numerical linear algebra routines to deliver both computational efficiency and stability because the update process needs to solve multiple regularized least-squares problems (Zhou et al., 2008).

The existing body of research on recommender systems has received extensive coverage, yet most investigations concentrate on developing model architectures and implementing regularization techniques and building feature engineering systems. The numerical methods which apply to ALS least-squares subproblem resolution have received

less research attention when compared to model architecture and regularization methods and feature engineering systems. Subproblems can use multiple linear algebra methods for solving which include explicit matrix inversion and direct factorization methods and iterative solvers.

The numerical linear algebra field treats these two options as different solutions. The traditional results demonstrate that explicit matrix inversion method operates at lower numerical stability which decreases its efficiency compared to linear system solving through factorization methods which include QR or Cholesky decomposition (Golub & Van Loan, 2013; Trefethen & Bau, 1997). The solver selection process affects both computational speed and the ability to reach a solution while maintaining system stability.

Small-to-medium benchmark datasets such as the MovieLens 100K dataset create a perfect testing ground to study these performance differences. The dataset enables controlled research which shows how solver selection affects outcomes because it is widely used and requires low computational power while all model components remain unchanged. The study of solver-level operations at this location delivers important findings which help both educational functions and actual implementations of recommender system operations.

Objectives

The main goal of this research is to analyze the performance of different linear system solvers in the context of Alternating Least Squares (ALS) for collaborative filtering. The specific objectives include:

1. Compare the computational efficiency of the following linear system solvers for solving the least-squares subproblems in ALS:
    1.1. Explicit Matrix inversion,
    1.2. Partial pivoting,
    1.3. LU decomposition,
    1.4. QR decomposition, and
    1.5. Cholesky decomposition
2. Evaluate the numerical stability of each solver by analyzing the condition number

of the matrices involved and the sensitivity of the solutions to perturbations in the input data.

3. Assess the impact of solver choice on the overall performance of the ALS algorithm in terms of convergence speed and the quality of the recommendations generated.

PRELIMINARIES

The necessary mathematical background for understanding the performance of linear system solvers in Alternating Least Squares (ALS) for collaborative filtering includes matrix factorization and numerical linear algebra techniques.

Collaborative Filtering and Matrix Factorization

Every day, millions of people receive personalized recommendations from movies on Netflix to products on Amazon without ever having to explicitly describe what they like. Behind many of these systems lies a simple but powerful idea: *people who agreed in the past are likely to agree in the future*. If two users have rated the same set of movies similarly, it is reasonable to assume that they share similar tastes, and that a movie one of them enjoyed might appeal to the other as well. This principle is the foundation of **collaborative filtering** (Ricci et al., 2015).

Collaborative filtering works purely from observed user behavior such as ratings, clicks, and purchases without needing to know anything about the items themselves. A streaming platform does not need to know that a film is a thriller or that it stars a particular actor; it only needs to know who watched it and how they rated it.

The Rating Matrix

The starting point for collaborative filtering is the **user-item rating matrix**. Imagine a large table where each row represents a user and each column represents an item. The entry at row i and column j, denoted R_{ij}, is the rating that user i gave to item j. The goal of a recommender system is to fill in the missing entries of this table to predict how a user would rate items they have not yet seen.

This matrix $R \in \mathbb{R}^{m \times n}$ is extremely sparse (mostly zeroes) with millions of users

and items, the vast majority of entries are unobserved. A typical user rates only a tiny fraction of all available items, which makes the prediction problem challenging.

Matrix Factorization

One of the most effective approaches to this problem is **matrix factorization**. The intuition is that user preferences and item characteristics can be described by a small number of underlying *latent factors*, hidden concepts such as genre, tone, or style that are not explicitly labeled but can be inferred from rating patterns. For instance, a user who consistently rates action movies highly implicitly reveals something about their taste, even if that preference is never explicitly stated.

Definition .1 (Matrix Factorization)**.** *Matrix factorization approximates the rating matrix R as the product of two smaller matrices:*

$$R \approx UV^\top$$

*where $U \in \mathbb{R}^{m \times f}$ is the **user factor matrix**, $V \in \mathbb{R}^{n \times f}$ is the **item factor matrix**, and $f < \min(m, n)$ is the number of latent factors.*

Each row $U_i \in \mathbb{R}^f$ is a compact numerical representation of user i's preferences, and each row $V_j \in \mathbb{R}^f$ captures the latent characteristics of item j. The predicted rating for user i on item j is the dot product:

$$\hat{R}_{ij} = U_i^\top V_j$$

It follows, if both the user vector and the item vector point in the same direction in the latent space, meaning the user tends to like what the item offers, the dot product will be large, indicating a strong predicted preference.

Learning the Factors

To find the best U and V, minimizing the difference between the predicted and actual ratings on all observed entries was done. A regularization term is added to prevent

overfitting, which is the tendency of a model to memorize training data rather than learning generalizable patterns. This leads to the following objective function:

$$\mathcal{L}(U, V) = \sum_{(i,j)\in\Omega} \left(R_{ij} - U_i^\top V_j\right)^2 + \lambda \left(\|U\|_F^2 + \|V\|_F^2\right)$$

where Ω is the set of observed ratings, $\lambda > 0$ is the regularization parameter that controls how strongly large factor values are penalized, and $\|\cdot\|_F$ is the Frobenius norm, which measures the overall magnitude of a matrix. The task of the ALS algorithm described in the next section is to minimize this objective efficiently (Koren et al., 2009).

Alternating Least Squares

Finding U and V that simultaneously minimize $\mathcal{L}(U, V)$ is difficult because the two unknowns are multiplied together, making the problem non-convex. However, there is creativite solution, if we temporarily fix one matrix and only optimize the other, the problem simplifies dramatically into a standard least-squares problem, which has a clean closed-form solution. **Altrrnating Least Squares (ALS)** does exactly this by repeatedly alternates between optimizing U with V held fixed, and optimizing V with U held fixed, until the solution converges (Zhou et al., 2008).

Think of it like adjusting two knobs on a stereo: instead of turning both at once, you fine-tune one while holding the other steady, then switch. Each individual adjustment is straightforward, and repeating this process gradually brings both knobs to their optimal positions.

User Factor Update

With V fixed, the problem of finding the best U_i for each user i becomes independent for each user, and reduces to minimizing:

$$\mathcal{L}(U_i) = \sum_{j\in\Omega_i} \left(R_{ij} - U_i^\top V_j\right)^2 + \lambda \|U_i\|^2$$

where Ω_i is the set of items that user i has rated. Setting the derivative of this expression to zero and solving gives the closed-form update:

$$\left(V_{\Omega_i}^\top V_{\Omega_i} + \lambda I\right) U_i = V_{\Omega_i}^\top R_{i,\Omega_i}$$

This is a system of linear equations of the form $Ax = b$, where $A = V_{\Omega_i}^\top V_{\Omega_i} + \lambda I$ and $b = V_{\Omega_i}^\top R_{i,\Omega_i}$. Solving it yields the updated latent factor vector for user i.

Item Factor Update

Similarly, fixing U and optimizing for each item j leads to an identical structure:

$$\left(U_{\Omega_j}^\top U_{\Omega_j} + \lambda I\right) V_j = U_{\Omega_j}^\top R_{\Omega_j,j}$$

where Ω_j is the set of users who have rated item j. Again, this is a linear system $Ax = b$ of the same form.

The Linear System under ALS

Both update steps reduce to solving a linear system:

$$Ax = b, \quad A = W^\top W + \lambda I \in \mathbb{R}^{f \times f}$$

where W is the relevant submatrix of either U or V. This system is small, the matrix A is only $f \times f$, where f is typically between 10 and 100, but it must be solved once for every user and once for every item at every iteration of ALS. With millions of users and items, this adds up quickly, making the efficiency of the chosen solver a significant factor in the overall runtime of the algorithm (Takács et al., 2009).

An important property of A is that it is always **symmetric positive definite** for any $\lambda > 0$. Symmetry means $A^\top = A$, and positive definiteness means that $x^\top Ax > 0$ for all nonzero vectors x, the matrix does not "collapse" any direction in space. This guarantees that the linear system has a unique solution at every step, and it also allows certain solvers, particularly Cholesky decomposition, to exploit this structure for improved speed and

stability. The full ALS procedure is summarized in Algorithm 1.

Figure 1

The ALS algorithm for matrix factorization.

Step 1. **Initialize** U and V with small random values.

Step 2. **Fix** V, solve for each row U_i by solving $\left(V_{\Omega_i}^\top V_{\Omega_i} + \lambda I\right) U_i = V_{\Omega_i}^\top R_{i,\Omega_i}$.

Step 3. **Fix** U, solve for each row V_j by solving $\left(U_{\Omega_j}^\top U_{\Omega_j} + \lambda I\right) V_j = U_{\Omega_j}^\top R_{\Omega_j,j}$.

Step 4. **Repeat** Steps 2–3 until the change in $\mathcal{L}$ falls below a threshold or a maximum number of iterations is reached.

System of Linear Equation Solvers

This section presents the five linear system solvers evaluated in this study. Each solver addresses the system $Ax = b$ where $A \in \mathbb{R}^{f\times f}$is the ALS subproblem matrix described above.

Explicit Matrix Inversion

The most straightforward approach is to compute A^{-1} explicitly and obtain the solution as:

$$x = A^{-1}b$$

Definition .2 (Matrix Inverse)**.** *For a nonsingular matrix $A \in \mathbb{R}^{n\times n}$, the inverse A^{-1} is the unique matrix satisfying $AA^{-1} = A^{-1}A = I$.*

The inverse can be computed using the adjugate formula:

$$A^{-1} = \frac{1}{\det(A)}\mathrm{adj}(A)$$

or more practically via row reduction of $[A \mid I]$. Computing A^{-1} requires$O(n^3)$ operations. While conceptually simple, explicit inversion accumulates rounding errors more aggressively than factorization-based methods, as each element of A^{-1} is formed from sums and products of all matrix entries (Anton & Rorres, 2014).

LU Decomposition

LU decomposition factors A into the product of a lower triangular matrix L and an upper triangular matrix U:

$$A = LU$$

Definition .3 (LU Decomposition)**.** *Given $A \in \mathbb{R}^{n\times n}$, LU decomposition finds $L, U \in \mathbb{R}^{n\times n}$ where L is unit lower triangular (diagonal entries equal to 1) and U is upper triangular, such that $A = LU$.*

The system $Ax = b$ is then solved in two triangular stages:

$$Ly = b \quad \text{(forward substitution)}$$
$$Ux = y \quad \text{(back substitution)}$$

Each triangular solve costs $O(n^2)$, and the factorization itself costs $O(n^3)$. The advantage of LU decomposition over explicit inversion is that the triangular structure is exploited directly, reducing numerical error propagation (Anton & Rorres, 2014).

Gaussian Elimination with Partial Pivoting

Gaussian elimination is a systematic method for solving a linear system $Ax = b$ by reducing the augmented matrix $[A \mid b]$ to upper triangular form through a sequence of row operations, after which the solution is recovered by back substitution. While straightforward in theory, a naive implementation can fail or produce highly inaccurate results when a pivot element, the leading entry used to eliminate entries below it, is zero or very small, causing division by a near-zero number and amplifying numerical errors. **Partial pivoting** addresses this by introducing a row-swapping strategy: before each elimination step, the algorithm scans the current column for the entry with the largest absolute value and swaps that row to the pivot position. This ensures that the largest available value is always used as the pivot, keeping the multipliers used during elimination bounded and preventing error amplification (Anton & Rorres, 2014).

Definition .4 (Gaussian Elimination with Partial Pivoting)**.** *At the k-th elimination step,*

partial pivoting selects the row $r \geq k$ with the largest absolute value in column k:

$$r = \arg\max_{k \leq i \leq n} |A_{ik}|$$

and swaps row r with row k before performing elimination. This is equivalent to factoring a row-permuted version of A:

$$PA = LU$$

where P is a permutation matrix recording all row swaps, L is unit lower triangular, and U is upper triangular.

The system $Ax = b$ is then solved in two stages using the factored form:

$$Ly = Pb \quad \text{(forward substitution)}$$

$$Ux = y \quad \text{(back substitution)}$$

The overall computational cost is $O(n^3)$ for the elimination phase, plus $O(n^2)$ for each triangular solve. Gaussian elimination with partial pivoting is the standard general-purpose method for solving dense linear systems and is the default approach in most numerical libraries precisely because it is robust across a wide range of matrix types without requiring any special structure (Anton & Rorres, 2014).

Cholesky Decomposition

When A is symmetric positive definite, as is the case for ALS subproblem matrices, Cholesky decomposition provides a more efficient and numerically stable factorization:

$$A = LL^\top$$

where L is a unique lower triangular matrix with positive diagonal entries.

Theorem .1 (Cholesky Decomposition)**.** *Every symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$ has a unique factorization $A = LL^\top$ where L is lower triangular with positive diagonal entries.*

The diagonal entries of L and the off-diagonal entries are computed column by column:

$$L_{kk} = \sqrt{A_{kk} - \sum_{j=1}^{k-1} L_{kj}^2}, \qquad L_{ik} = \frac{1}{L_{kk}} \left(A_{ik} - \sum_{j=1}^{k-1} L_{ij} L_{kj} \right), \quad i > k$$

The system is then solved via forward and back substitution:

$$Ly = b$$

$$L^\top x = y$$

Cholesky decomposition requires approximately $\frac{1}{3}n^3$ floating-point operations, roughly half the cost of standard LU decomposition and is the recommended solver when the matrix is known to be symmetric positive definite (Axler, 2024).

QR Decomposition

QR decomposition factors A into an orthogonal matrix Q and an upper triangular matrix R:

$$A = QR$$

Definition .5 (QR Decomposition)**.** *For $A \in \mathbb{R}^{n \times n}$, QR decomposition finds an orthogonal matrix Q (satisfying $Q^\top Q = I$) and an upper triangular matrix R such that $A = QR$.*

QR decomposition is typically computed via Householder reflections or Givens rotations. The solution to $Ax = b$ is obtained as:

$$x = R^{-1} Q^\top b$$

which is computed by first forming $c = Q^\top b$ and then solving the triangular system $Rx = c$ by back substitution. QR decomposition costs $O(2n^3/3)$ operations for a square matrix and is numerically very stable because orthogonal transformations do not amplify errors (Axler, 2024).

METHODS AND APPLICATIONS

ANALYSIS AND RESULTS

CONCLUSION

REFERENCES

Anton, H., & Rorres, C. (2014). *Elementary linear algebra: Applications version* (11th ed.). John Wiley & Sons, Inc.

Axler, S. (2024). *Linear algebra done right* (4th ed.). Springer.

Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (4th ed.). Johns Hopkins University Press.

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, *42*(8), 30–37. https://doi.org/10.1109/MC.2009.263

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). *Recommender systems handbook* (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6

Takács, G., Pilászy, I., Németh, B., & Tikk, D. (2009). Scalable collaborative filtering approaches for large recommender systems. *Journal of Machine Learning Research*, *10*, 623–656. http://jmlr.org/papers/v10/takacs09a.html

Trefethen, L. N., & Bau, D. (1997). *Numerical linear algebra*. SIAM. https://doi.org/10.1137/1.9781611971484

Zhou, Y., Wilkinson, D., Schreiber, R., & Pan, R. (2008). Large-scale parallel collaborative filtering for the netflix prize. *Proceedings of the AAAI Conference on Artificial Intelligence*, 337–348.